50

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

FluxCD Mimarisi: 4 Silahşörler Kontrolcüleri

FluxCD Architecture: The Four Musketeer Controllers

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Flux devasa, hantal tek bir program değildir; her biri kendi işinde uzmanlaşmış 4 süper kahraman robottur: 1) Source Controller: Gözlerini kırpmadan Git deposunu ve Docker registry'yi izleyen gözcü. 2) Kustomize Controller: Çekilen YAML'ları K8s kümesine uygulayan usta işçi. 3) Helm Controller: Dışarıdan paket indiren kargo şefi. 4) Notification Controller: Bir şey olduğunda Slack veya Teams'e haber uçuran haberci!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Flux is not a monolithic bloated daemon; it is a modular league of 4 specialized micro-controllers: 1) Source Controller: The tireless scout watching Git repos and OCI registries. 2) Kustomize Controller: The builder reconciling raw manifests into the cluster. 3) Helm Controller: The logistics manager orchestrating Helm chart lifecycles. 4) Notification Controller: The town crier alerting Slack/Teams when events occur!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Flux v2 (GitOps Toolkit) mimarisi Kubernetes Controller Runtime prensiplerine tam uyumludur: 1) `source-controller`: Git depolarını, OCI imajlarını, Helm repo'larını klonlar ve bir gzip tarball olarak küme içi belleğe sunar. 2) `kustomize-controller`: Bu paketi alır, Kustomize yamalarını (overlays) çözer, K8s API'sine uygular ve `prune: true` ile Git'ten silinen kaynakları kümeden de siler. 3) `helm-controller`: Helm paketlerinin sürümlerini ve yükseltmelerini bildirici yönetir. 4) `notification-controller`: Webhook'ları kabul eder.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The Flux GitOps Toolkit is a composable set of specialized controllers: 1) `source-controller` clones Git/Helm/Bucket repositories and serves them as localized tarball artifacts. 2) `kustomize-controller` evaluates manifests via Kustomize engines, applies resources to the API Server, and garbage collects orphaned resources when deleted from Git (`prune: true`). 3) `helm-controller` manages Helm chart releases declaratively. 4) `notification-controller` manages outbound notifications and inbound webhooks.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Flux Kontrolcülerinin Kümedeki Durumunu Sorgula:
flux check

# Çıktı:
# ► checking controllers
# ✔ source-controller: deployment ready
# ✔ kustomize-controller: deployment ready
# ✔ helm-controller: deployment ready
# ✔ notification-controller: deployment ready
# ✔ all checks passed
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Eski Flux v1 tek parçaydı (monolith). Modern Flux v2 ise modüler 'GitOps Toolkit' üzerine kuruludur. Eski v1 dokümanlarındaki `fluxctl` komutlarını kullanmayın; daima modern `flux` CLI'ını kullanın!

🇬🇧 WATCH OUT

Never use legacy Flux v1 tutorials or the deprecated `fluxctl` CLI. Modern Flux v2 is entirely re-architected around the GitOps Toolkit controller family.

🎯 Özet Çıkarımlar

- Flux modüler yapısı sayesinde sadece ihtiyaç duyulan kontrolcüler kurulabilir.
- Kustomize Controller otomatik çöp toplayıcı (Garbage Collector) gibi çalışır.
- Notification Controller ile dağıtım durumları Slack veya Discord'a bağlanır.

🎯 Key Takeaways

- Micro-controller architecture allows adopting only the controllers you actually need.
- Kustomize controller's `prune: true` automatically purges deleted resources cleanly.
- Notification controller routes release events directly to corporate communication channels.